

Course Outline: Managing Related Tables

Title: Managing Related Tables with SQL **Skill:** Skill 31 — Consultar tablas relacionadas con SQL
Learnpack: + Gestionando tablas relacionadas con SQL **File:** s31-w11d32-gestionando-tablas-relacionadas-con-sql **Prerequisite:** s30-w11d31-bases-de-datos (SQL fundamentals, SELECT/INSERT/UPDATE/DELETE, Supabase) **Target Audience:** Beginner developers who know basic SQL and want to model and query real-world data across multiple related tables **Total Lessons:** 12
Format: Mostly Conceptual (8% hands-on — 1 code challenge)

Course Description

You already know how to talk to a single table. But real applications never live in a single table — users have orders, orders have products, products have categories. This course teaches you how to model those real-world relationships in a relational database and how to query across them using SQL JOINS. By the end, you will be able to design a multi-table schema using foreign keys, choose the right relationship type (1:1, 1:n, or n:m), and retrieve combined results from two or more related tables using INNER JOIN, LEFT JOIN, RIGHT JOIN, and OUTER JOIN.

Course Philosophy

This course emphasizes:

- **Starting from the problem, not the syntax** — students understand WHY relations exist before learning how to implement them
 - **Designing before querying** — schema design (E/R diagrams, relation types, pivot tables) comes before JOIN syntax so queries feel like a natural consequence, not magic
 - **Observing before abstracting** — the ORM abstraction (coming in Skill 33) will make more sense once students have seen the raw SQL underneath it
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Why Tables Relate	2
02 - Designing Relationships	2
03 - Querying Related Data	3
04 - Responsible Design	2
05 - Assessment	1
06 - Outro	1

Section	Lessons
Total	12

Section 00: Welcome

00.0 Welcome to Related Tables

Learning Objectives:

- Understand what this course covers and why it matters
- Identify the gap between a single-table database and a real application's needs
- Preview the skills covered: E/R diagrams, foreign keys, relation types, JOINS, normalization

Content Outline:

- The problem with flat tables: why a single table breaks down in real applications
- What students will be able to do by the end: design multi-table schemas and query them with JOINS
- Brief section overview: design first, query second, design responsibly third

Transitions: Leads into Section 01, which shows exactly what breaks when you try to fit everything into one table.

Key Principles:

- Real-world data is naturally relational — tables model entities, not everything
- Designing a good schema before writing queries saves hours of refactoring later

Examples:

- A simple e-commerce scenario: users, orders, and products — why you can't flatten this into one table without repeating data
 - A school database: students, courses, and enrollments — three distinct entities, three distinct tables
-

Section 01: Why Tables Relate

01.0 When One Table Isn't Enough

Learning Objectives:

- Explain what a relation is in the context of a relational database
- Identify the problems caused by storing everything in a single flat table (data redundancy, update anomalies)
- Describe how splitting data into related tables solves those problems

Content Outline:

- What is a relation in a relational database? (A structured link between two entities)
- The flat-table trap: what happens when you put a user's city and their orders in the same row
 - Data duplication: the city appears in every order row for the same user
 - Update anomalies: change the city in one row, forget another
 - Deletion anomalies: delete the last order, lose the user's data
- The solution: separate entities into separate tables and link them
- Relations as pointers: one table "points to" a row in another

Transitions: Now that we know WHY we need relations, the next lesson introduces the standard tool for designing them before writing any SQL: the Entity-Relationship diagram.

Key Principles:

- A table represents one entity — not a mix of entities
- Data should be stored once and referenced many times, not duplicated
- Relations solve redundancy by linking tables instead of embedding data

Examples:

- `users` table (id, name, city) + `orders` table (id, user_id, total) — the city is stored once in `users`, referenced by `user_id` in `orders`
 - Counter-example: a flat `user_orders` table where name and city repeat for every order
-

01.1 Entity-Relationship Diagrams

Learning Objectives:

- Define what an Entity-Relationship (E/R) diagram is and what it represents
- Map real-world elements (people, things, events) to entities and attributes in an E/R diagram
- Read a simple E/R diagram and describe the entities and their relationships

Content Outline:

- What is an E/R diagram? A visual blueprint of a database schema
- Three components: entities (boxes), attributes (fields inside entities), and relationships (lines between entities)
- How to identify entities: each distinct "thing" in the system becomes a table
- How to identify attributes: each property of that thing becomes a column
- How to identify relationships: when two entities interact or reference each other, draw a line
- Reading cardinality notation: 1, n (many), m (many on both sides)
- E/R diagrams as a design tool — you draw them before writing a single line of SQL

Transitions: The E/R diagram shows us WHAT relationships exist and their type. The next section shows us HOW to implement those relationships in actual SQL: with foreign keys.

Key Principles:

- Design the schema visually before coding it — this prevents expensive restructuring later

- Every entity in the diagram becomes a table; every relationship becomes a foreign key (or a pivot table for n:m)
- The E/R diagram is a communication tool — it lets the team agree on the data model before implementation

Examples:

- E/R diagram for a blog: **users** entity (id, name, email), **posts** entity (id, title, body, published_at), relationship: a user writes many posts
- E/R diagram for an online store: **products**, **orders**, **customers** — identifying which pairs have relationships and what kind

Section 02: Designing Relationships

02.0 Foreign Keys

Learning Objectives:

- Define what a foreign key is and its role in a relational database
- Explain how a foreign key creates a link between two tables
- Describe what happens when a foreign key constraint is violated

Content Outline:

- What is a foreign key? A column in one table that holds the primary key of a row in another table
- How it works: the **user_id** column in **orders** references the **id** column in **users**
- The constraint: PostgreSQL enforces referential integrity — you cannot insert an **order** with a **user_id** that doesn't exist in **users**
- Naming convention: **[referenced_table_singular]_id** (e.g., **user_id**, **product_id**)
- The difference between the owning side and the referenced side

Transitions: Foreign keys are the mechanism. The next lesson covers what kinds of relationships they create — one-to-one, one-to-many, and many-to-many — and what each looks like in practice.

Key Principles:

- A foreign key is always a column in the "many" side of a 1:n relationship
- Referential integrity means the database actively prevents orphaned records
- Naming foreign keys consistently (**table_id**) makes schemas readable

Examples:

```
-- users table (referenced)
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL
);

-- orders table (owns the foreign key)
```

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id),  
  total NUMERIC NOT NULL  
);
```

- Attempting to insert `INSERT INTO orders (user_id, total) VALUES (999, 50.00)` when user 999 doesn't exist → PostgreSQL raises a foreign key violation error
-

02.1 Relation Types and Pivot Tables

Learning Objectives:

- Distinguish between 1:1, 1:n, and n:m relationship types with concrete examples
- Implement a 1:n relationship using a foreign key
- Explain what a pivot table is and why it's needed for n:m relationships

Content Outline:

- 1:1 relationship: each row in Table A corresponds to exactly one row in Table B (and vice versa)
 - Example: `users` and `user_profiles` — each user has exactly one profile
 - Implementation: foreign key in either table, with a UNIQUE constraint
- 1:n relationship: one row in Table A corresponds to many rows in Table B
 - Example: one user has many orders — the foreign key lives in `orders`
 - This is the most common relationship type
- n:m relationship: many rows in Table A correspond to many rows in Table B
 - Example: students and courses — a student enrolls in many courses, a course has many students
 - Cannot be implemented with just a foreign key in either table
- Pivot table (junction table): a third table that holds the relationship between two n:m entities
 - Contains foreign keys to both tables
 - May contain additional attributes about the relationship itself (e.g., `enrolled_at`)

Transitions: Now we know how to design a schema with related tables. The next section teaches how to query across those relationships — pulling data from two or more tables into a single result using JOINS.

Key Principles:

- The foreign key always lives on the "many" side: in 1:n, it's in the "many" table; in n:m, it's in the pivot table
- A pivot table is not a workaround — it's the correct, standard implementation for n:m
- The pivot table can hold meaningful data about the relationship itself (timestamp, status, role)

Examples:

- 1:1: `users` (id, email) ↔ `user_profiles` (id, user_id UNIQUE, bio, avatar_url)
- 1:n: `authors` (id, name) → `books` (id, author_id, title)

- n:m via pivot: `students` ↔ `enrollments` ↔ `courses` — `enrollments` has `student_id` and `course_id` as foreign keys
-

Section 03: Querying Related Data

03.0 INNER JOIN

Learning Objectives:

- Explain what INNER JOIN does and when to use it
- Write a SELECT query with INNER JOIN to retrieve data from two related tables
- Interpret the result set of an INNER JOIN query

Content Outline:

- The core problem: SELECT from one table only returns columns from that table — how do you get columns from two tables at once?
- What INNER JOIN does: combines rows from two tables where the join condition is TRUE — rows with no match on either side are excluded
- Syntax: `SELECT ... FROM table_a INNER JOIN table_b ON table_a.id = table_b.table_a_id`
- The ON clause: specifies the columns that link the two tables (usually the foreign key = primary key)
- Table aliases for readability: `FROM users u INNER JOIN orders o ON u.id = o.user_id`
- What happens to unmatched rows: they disappear from the result — INNER JOIN is strict

Transitions: INNER JOIN is the most common join — but it excludes any row with no match. The next lesson covers the outer join family, which lets you keep unmatched rows.

Key Principles:

- INNER JOIN returns only rows where the match condition is satisfied on BOTH sides
- Always use aliases when joining multiple tables — it prevents ambiguous column references
- Use `table.column` notation in the SELECT list when the same column name exists in both tables

Examples:

```
-- Get all orders with the name of the user who placed them
SELECT u.name, o.id AS order_id, o.total
FROM users u
INNER JOIN orders o ON u.id = o.user_id;
-- Result: only users who have at least one order appear; users with no
orders are excluded
```

- Counter-example: a user who has never placed an order does NOT appear in the result — if you need them, use LEFT JOIN
-

03.1 Outer Joins

Learning Objectives:

- Distinguish between LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN
- Identify when each outer join type is appropriate
- Read and interpret NULL values in outer join results

Content Outline:

- The problem INNER JOIN doesn't solve: what if you need ALL rows from one table, whether or not they have a match?
- LEFT JOIN: returns all rows from the LEFT table, and matched rows from the right — unmatched right-side rows appear as NULL
 - Use case: "show me all users, including those who have never placed an order"
- RIGHT JOIN: the mirror of LEFT JOIN — all rows from the RIGHT table, matched rows from the left
 - Use case: less common; usually rewritten as a LEFT JOIN with tables swapped
- FULL OUTER JOIN: all rows from BOTH tables — unmatched rows on either side appear as NULL
 - Use case: auditing or reconciliation — "show me everything, even orphaned rows"
- Reading NULL in join results: a NULL in a right-side column means the left row had no match
- Practical tip: LEFT JOIN is the most commonly used outer join; RIGHT JOIN and FULL OUTER JOIN are rare but important to recognize

Transitions: The next lesson puts this knowledge into practice — write a JOIN query yourself and observe the result set directly.

Key Principles:

- Outer joins preserve unmatched rows; INNER JOIN discards them
- NULL in a join result means "no matching row was found on that side"
- LEFT vs RIGHT is about which table you want to keep whole — choose based on which side is your "anchor"

Examples:

```
-- LEFT JOIN: all users, even those with no orders
SELECT u.name, o.id AS order_id
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
-- Users with no orders appear with order_id = NULL

-- FULL OUTER JOIN: all users and all orders, matched or not
SELECT u.name, o.id AS order_id
FROM users u
FULL OUTER JOIN orders o ON u.id = o.user_id;
```

- Anti-pattern: using FULL OUTER JOIN when you only need LEFT JOIN — produces unnecessary rows and confuses the intent

03.2 Query Across Tables

Challenge Prompt: Given a **books** table (id, title, author_id) and an **authors** table (id, name), write a query that returns the title of each book alongside the name of its author, including books that have no author assigned.

Solution Code:

```
SELECT b.title, a.name AS author_name
FROM books b
LEFT JOIN authors a ON b.author_id = a.id;
```

Challenge Code:

```
-- books: id (SERIAL PK), title (TEXT), author_id (INTEGER, nullable FK →
authors.id)
-- authors: id (SERIAL PK), name (TEXT)

-- Write a query that returns each book's title and its author's name.
-- Books with no author assigned should still appear (with author_name as
NULL).

SELECT -- your code here
FROM -- your code here
```

Section 04: Responsible Design

04.0 Normalization

Learning Objectives:

- Define database normalization and explain why it is advisable
- Identify common signs that a schema is not normalized (data duplication, update anomalies)
- Describe the practical benefit of normalizing a schema

Content Outline:

- What is normalization? The process of structuring a database to reduce data redundancy and improve data integrity
- The core idea: each piece of information should live in exactly one place
- Why it matters: duplicate data leads to inconsistencies — update one copy, forget another
- First Normal Form (1NF) — practical introduction: no repeating groups, each column holds one value
- Second Normal Form (2NF): every non-key column depends on the whole primary key (relevant when composite keys are used)

- Third Normal Form (3NF): no column depends on another non-key column (no transitive dependencies)
- You don't need to memorize the forms — you need to feel when something is duplicated unnecessarily
- When to denormalize: performance-critical read-heavy systems sometimes store redundant data intentionally — but this is an advanced trade-off, not a beginner default

Transitions: The last concept in this course is cascade delete — a SQL mechanism that handles what happens to related rows when a parent row is deleted. This is informational: important to know it exists, not required to implement today.

Key Principles:

- Normalized schemas are easier to maintain — change data once, not everywhere
- Normalization is the default; denormalization is an optimization with trade-offs
- Well-designed foreign key relationships are the backbone of a normalized schema

Examples:

- Unnormalized: a `orders` table with `customer_name`, `customer_email`, `customer_city` repeated in every row → if the customer moves, you update hundreds of rows
- Normalized: `customers` table holds the customer data once; `orders` references `customer_id` → update one row in `customers`, all orders reflect the change
- Real-world benefit: a product price change in an e-commerce system updates one row in `products`, not thousands of `order_items` rows

04.1 Cascade Delete

Learning Objectives:

- Explain what cascade delete is and when PostgreSQL triggers it
- Describe the risk of cascade delete and why it must be used deliberately
- Identify the SQL syntax for enabling cascade delete on a foreign key

Content Outline:

- The problem: if you delete a row in `users`, what happens to that user's rows in `orders`?
 - Default behavior: PostgreSQL raises a foreign key violation error — it refuses to delete the parent row while children exist
- What is cascade delete? A setting on the foreign key that tells PostgreSQL: "if the parent is deleted, automatically delete all its children too"
- The SQL syntax: `REFERENCES users(id) ON DELETE CASCADE`
- When it's appropriate: data with a clear ownership hierarchy where the children have no meaning without the parent (e.g., a shopping cart's items when the cart is deleted)
- When it's dangerous: any time the child data has independent value or is referenced by audit logs, analytics, or other systems
- The safe default: do NOT use cascade delete unless you have a specific reason — let PostgreSQL's default protection prevent accidental data loss

- This is informational — you will not implement cascade delete in this course, but you will encounter it in production schemas

Transitions: This completes the course content. The assessment section tests everything covered: relations, schema design, JOINS, normalization, and cascade delete.

Key Principles:

- Cascade delete is a power tool — it can silently delete large amounts of data
- Always know what "children" a row has before deciding whether cascade delete is safe
- Prefer explicit application-level deletes over cascade delete for important data

Examples:

```
-- With cascade: deleting a user automatically deletes all their orders
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  total NUMERIC NOT NULL
);

-- Without cascade (default): attempting to delete a user who has orders
-- raises an error
-- DELETE FROM users WHERE id = 1; → ERROR: update or delete on table "users"
-- violates foreign key constraint
```

- Anti-pattern: enabling **ON DELETE CASCADE** everywhere "for convenience" — this is how production databases accidentally lose critical data

Section 05: Assessment

05.0 Knowledge Check 🧠

Learning Objectives:

- Demonstrate understanding of relational database concepts: relations, foreign keys, relation types
- Identify the correct JOIN type for a given query requirement
- Recognize normalized vs. unnormalized schemas
- Apply understanding of E/R diagrams to real-world data modeling scenarios

Question Format: Scenario-based multiple choice

Questions:

1. You have a **students** table and a **courses** table. Many students can enroll in many courses. Which of the following correctly models this relationship?
 - a) Add a **course_id** column directly to **students**

- b) Add a `student_id` column directly to `courses`
 - c) Create a pivot table `enrollments` with `student_id` and `course_id` as foreign keys ✓
 - d) Combine students and courses into one table
2. You run the following query: `SELECT u.name, o.id FROM users u INNER JOIN orders o ON u.id = o.user_id`. A user named "Alice" exists but has no orders. What happens?
- a) Alice appears with `o.id` as NULL
 - b) Alice does not appear in the result ✓
 - c) The query raises an error
 - d) Alice appears once for every row in the `orders` table
3. Which JOIN type should you use to retrieve all rows from the left table, including those with no matching row in the right table?
- a) INNER JOIN
 - b) LEFT JOIN ✓
 - c) RIGHT JOIN
 - d) FULL OUTER JOIN
4. You want to list all products, including products that have never appeared in any order. Which query is correct?
- a) `SELECT p.name FROM products p INNER JOIN order_items oi ON p.id = oi.product_id`
 - b) `SELECT p.name FROM products p LEFT JOIN order_items oi ON p.id = oi.product_id` ✓
 - c) `SELECT p.name FROM products p RIGHT JOIN order_items oi ON p.id = oi.product_id`
 - d) `SELECT p.name FROM products p, order_items oi WHERE p.id = oi.product_id`
5. What is a foreign key?
- a) A column that holds a unique identifier for its own table
 - b) A column in one table that references the primary key of another table ✓
 - c) A column used to sort query results
 - d) A column that can never be NULL
6. A `books` table has columns: `id`, `title`, `author_name`, `author_country`, `author_birth_year`. Why is this problematic?
- a) A book cannot have an author
 - b) Author data is duplicated for every book by the same author, causing update anomalies ✓
 - c) Books should not have a title column
 - d) This is perfectly fine — flat tables are always better
7. In a 1:n relationship between `departments` and `employees`, where should the foreign key live?

- a) In **departments**, pointing to **employees**
 - b) In **employees**, pointing to **departments** ✓
 - c) In a separate pivot table
 - d) In both tables simultaneously
8. You delete a row from **users**. PostgreSQL raises a foreign key violation error. What does this tell you?
- a) The **users** table has no primary key
 - b) There are rows in another table that reference this user, and no cascade delete is set ✓
 - c) The **users** table is read-only
 - d) The user does not exist
9. What does **ON DELETE CASCADE** do on a foreign key?
- a) Prevents the parent row from being deleted
 - b) Automatically deletes all child rows when the parent row is deleted ✓
 - c) Sets child foreign key values to NULL when the parent is deleted
 - d) Copies the parent row before deleting it
10. An E/R diagram shows: **authors** — 1:n — **books** — n:m — **genres**. How many tables are needed to implement this schema?
- a) 2
 - b) 3
 - c) 4 ✓ (**authors**, **books**, **genres**, and a pivot table **book_genres**)
 - d) 5

Evaluation Criteria:

- 9-10 correct: Excellent command of relational database design and SQL JOINS
- 7-8 correct: Solid understanding; revisit the JOIN types or normalization lessons
- 5-6 correct: Partial understanding; review Sections 02 and 03 before moving on
- Below 5: Return to Section 01 and work through the course again

Score Interpretation: A passing score (7+) indicates readiness to move on to Skill 32 (OOP in Python) and later Skill 33 (ORM with SQLAlchemy, which will build directly on the relational concepts learned here).

Section 06: Outro

06.0 What You've Built

Content Outline:

- Course recap: what students accomplished
 - You can now design a multi-table schema using E/R diagrams
 - You understand foreign keys, relation types (1:1, 1:n, n:m), and pivot tables

- You can query across related tables using INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN
- You know what normalization is and why it matters
- You understand cascade delete and when NOT to use it
- Connection to bigger picture
 - Relational databases are the backbone of most production applications — everything you build from here will likely have multiple related tables
 - In Skill 33, you will learn SQLAlchemy ORM, which generates these JOINS automatically — but now you understand what it's doing under the hood
 - The schema design skills from this course apply to every database, not just PostgreSQL
- Next steps and continued learning
 - Practice: try designing a schema for a project you care about before looking at code
 - Explore: tools like dbdiagram.io let you draw E/R diagrams online and export SQL
 - Challenge yourself: write a 3-table JOIN query (two joins in one SELECT)
- Resources for further exploration
 - PostgreSQL docs on joins: postgresql.org/docs/current/queries-table-expressions.html
 - Use the Supabase table editor (from the previous learnpack) to visualize your tables and their relationships
- Encouragement: You've just crossed a major milestone. Single-table SQL is a skill — multi-table SQL is a superpower.

Note: This is conclusion only — no theory, exercises, or assessment.